Базовые алгоритмы теории чисел

План

- НОД/НОК
- Проверка на простоту
- Нахождение делителей
- Нахождение простых делителей

Наибольший общий делитель

Наибольшим общим делителем целых чисел a и b называют такое число d, что a и b делятся на d, при этом d – наибольшее.

Алгоритм Евклида

Алгоритм Евклида

- Используем следующие свойства НОД:

Алгоритм Евклида

- Используем следующие свойства НОД:

НОД(a, b) = a, a = b

НОД(a, b) = НОД(a – b, b), a > b

НОД(a, b) = НОД(b, a), a < b

Реализация

C++	Python	Pascal

`while (a != b){`
`  if (a > b){`
`  a = a - b;`
`  }`
`  else {`
`  swap(a, b);`
`  }`
`}`
	`while a != b:`
	`  if a > b:`
	`  a -= b`
	`  else:`
	`  a, b = b, a`
		`while (a <> b) do begin`
		`  if (a > b) then`
		`  a := a - b`
		`  else begin`
		`  t := a;`
		`  a := b;`
		`  b := t;`
		`  end;`
		`end;`

Основные целочисленные типы данных

C++	Python	Pascal
int: $-2^{31}..2^{31}-1$ long long: $-2^{63}..2^{63}-1$	int: нет ограничений на значения (тем больше число, тем медленнее оно обрабатывается)	int: $-2^{31}..2^{31}-1$ long long: $-2^{63}..2^{63}-1$

Проблема

- Используем следующие свойства НОД:

НОД(a, b) = a, a = b

НОД(a, b) = НОД(a – b, b), a > b

НОД(a, b) = НОД(b, a), a < b

Проблема

- Используем следующие свойства НОД:

НОД(a, b) = a, a = b

НОД(a, b) = НОД(a – b, b), a > b

НОД(a, b) = НОД(b, a), a < b

Что, если a = 0 или b = 0?

Проблема

- Используем следующие свойства НОД:

НОД(a, b) = a, a = b

НОД(a, b) = НОД(a – b, b), a > b

НОД(a, b) = НОД(b, a), a < b

Что, если a = 0 или b = 0?

Вычитания будут происходить бесконечно.

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(a – b, b), a ≥ b

НОД(a, b) = НОД(b, a), a < b

Алгоритм Евклида

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(a – b, b), a ≥ b

НОД(a, b) = НОД(b, a), a < b

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления.

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(a mod b, b), a ≥ b

НОД(a, b) = НОД(b, a), a < b

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления.

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(a mod b, b), a ≥ b

НОД(a, b) = НОД(b, a), a < b

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления. Упакуем 2 и 3 свойства в одно.

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(b, a mod b)

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления. Упакуем 2 и 3 свойства в одно.

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(b, a mod b)

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления. Упакуем 2 и 3 свойства в одно.

Если a < b

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(b, a mod b)

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления. Упакуем 2 и 3 свойства в одно.

Если a < b, то a mod b = a

Коррекция

- Используем следующие свойства НОД:

НОД(a, 0) = a

НОД(a, b) = НОД(b, a mod b)

Вычитание b из a до тех пор, пока a не станет меньше b, аналогично взятию остатка от деления. Упакуем 2 и 3 свойства в одно.

Если a < b, то a mod b = a, то есть переменные поменяются местами

Реализация

Реализация

C++	Python	Pascal

`while (b != 0){`
`  a %= b;`
`  swap(a, b);`
`}`
	`while b != 0:`
	`  a %= b`
	`  a, b = b, a`
		`while (b <> 0) do begin`
		`  a := a mod b;`
		`  t := a;`
		`  a := b;`
		`  b := t;`
		`end;`

Наименьшее общее кратное

Наименьшее общее кратное чисел a и b – минимальное число, которое делится и на a, и на b.

Свойства НОД и НОК

Наименьшее общее кратное чисел a и b – минимальное число, которое делится и на a, и на b.

a * b = НОД(a, b) * НОК(a, b)

Свойства НОД и НОК

Наименьшее общее кратное чисел a и b – минимальное число, которое делится и на a, и на b.

a * b = НОД(a, b) * НОК(a, b)

НОК(a, b) = a * b / НОД(a, b)

Простые числа

Простые числа

Число называют простым, оно имеет ровно 2 различных натуральных делителя: единицу и само себя.

Простые числа

Число называют простым, оно имеет ровно 2 различных натуральных делителя: единицу и само себя.

Важный момент

Простые числа

Число называют простым, оно имеет ровно 2 различных натуральных делителя: единицу и само себя.

Важный момент: единица простым числом не является!

Наивная проверка на простоту

Наивная проверка на простоту

Проверим число N на простоту.

Наивная проверка на простоту

Проверим число N на простоту.

Пробежимся по всем числам от 1 до N,

проверим, делится ли N на каждое из чисел.

Наивная проверка на простоту

Проверим число N на простоту.

Пробежимся по всем числам от 1 до N, проверим, делится ли N на каждое из чисел.

Если мы нашли только 2 делителя, число простое, иначе – нет.

Реализация

C++	Python	Pascal

`cnt = 0;`
`for(int i = 1; i <= N; ++i){`
`  if (N % i == 0){`
`  ++cnt;`
`  }`
`}`
`if (cnt == 2){`
`  cout << "YES";`
`}`
`else {`
`  cout << "NO";`
`}`
	`cnt = 0`
	`for i in range (1, N + 1):`
	`  if N % i == 0:`
	`  cnt += 1`
	`if cnt == 2:`
	`  print('YES')`
	`else:`
	`  print('NO')`
		`cnt := 0`
		`for i := 1 to N do begin`
		`  if (N mod i = 0) then`
		`  cnt := cnt + 1;`
		`end;`
		`if (cnt = 2) then`
		`  write('YES')`
		`else`
		`  write('NO')`

В чем проблема?

Возьмём большое простое число (например, 1000000007)

В чем проблема?

Возьмём большое простое число (например, 1000000007). Чтобы проверить его на простоту, придётся пробежаться по всем числам от 1 до 1000000007.

Как ускорить?

Как ускорить?

Рассмотрим следующее утверждение:

Как ускорить?

Рассмотрим следующее утверждение:

Число N (N > 1) не будет являться простым, если у него есть делитель, не больший корня из этого числа ($\sqrt{N}$) и не равный единице.

Доказательство

Метод от противного.

Пусть N не является простым числом и всего его делители больше $\sqrt{N}$. Тогда у него есть делитель $x > \sqrt{N}$. Но тогда если N делится на x, то оно делится и на $\frac{N}{x}$, причём $\frac{N}{x} > \sqrt{N}$. Но тогда $N = x \cdot \frac{N}{x} > \sqrt{N} \cdot \sqrt{N} = N$!!!

Как применить в проверке на простоту

Заведём флаг, который будет сообщать, есть ли у числа делитель не больше его корня.

Как применить в проверке на простоту

Заведём флаг, который будет сообщать, есть ли у числа делитель не больше его корня. Пробежимся от 2 до корня из числа.

Как применить в проверке на простоту

Заведём флаг, который будет сообщать, есть ли у числа делитель не больше его корня.

Пробежимся от 2 до корня из числа.

Стоит отдельно рассмотреть единицу.

Как пробежаться до корня?

Заведём флаг, который будет сообщать, есть ли у числа делитель не больше его корня.

Пробежимся от 2 до корня из числа.

Стоит отдельно рассмотреть единицу.

Как пробежаться до корня?

Заведём флаг, который будет сообщать, есть ли у числа делитель не больше его корня.

Пробежимся от 2 до корня из числа.

Стоит отдельно рассмотреть единицу.

Чтобы пробежаться до корня, поставим условие: $i^2 \leq N$

Реализация

```
flag := 1;
i := 2;
while (sqr(i) <= N) do begin
    if (N mod i = 0) then begin
        flag := 0;
        break;
    end;
    inc(i);
end;
if (N = 1) then
    flag := 0;
if (flag = 1) then
    write('YES')
else
    write('NO')
```

Реализация

```
flag = 1
i = 2
while i ** 2 <= N:
    if N % i == 0:
        flag = 0
        break
    i += 1
if N == 1:
    flag = 0
if flag == 1:
    print('YES')
else:
    print('NO')
```

Реализация

```
flag = 1;
for(int i = 2; i * i <= N; ++i){
    if (N % i == 0){
        flag = 0;
        break;
    }
}
if (N == 1){
    flag = 1;
}
if (flag == 1){
    cout << "YES";
}
else {
    cout << "NO";
}
```

Пробег по простым в отрезке

```
for N := l to r do begin
    flag := 1;
    i := 2;
    while (sqr(i) <= N) do begin
        if (N mod i = 0) then begin
            flag := 0;
            break;
        end;
        inc(i);
    end;
    if (N = 1) then
        flag := 0;
    if (flag = 1) then
        writeln('YES')
    else
        writeln('NO')
end;
```

Пробег по простым в отрезке

```
for N in range(l, r + 1):
    flag = 1
    i = 2
    while i ** 2 <= N:
        if N % i == 0:
            flag = 0
            break
        i += 1
    if N == 1:
        flag = 0
    if flag == 1:
        print('YES')
    else:
        print('NO')
```

Пробег по простым в отрезке

```
for(int N = l; N <= r; ++N){
    flag = 1;
    for(int i = 2; i * i <= N; ++i){
        if (N % i == 0){
            flag = 0;
            break;
        }
    }
    if (N == 1){
        flag = 1;
    }
    if (flag == 1){
        cout << "YES" << endl;
    }
    else {
        cout << "NO" << endl;
    }
}
```

Поиск делителей числа

Воспользуемся идеей, похожей на проверку числа на простоту.

Поиск делителей числа

Воспользуемся идеей, похожей на проверку числа на простоту.

Пробежимся по всем i от 1 до корня из числа.

Поиск делителей числа

Воспользуемся идеей, похожей на проверку числа на простоту.

Пробежимся по всем i от 1 до корня из числа.

Если число N делится на i, то оно делится и на N / i

Поиск делителей числа

Воспользуемся идеей, похожей на проверку числа на простоту.

Пробежимся по всем i от 1 до корня из числа.

Если число N делится на i, то оно делится и на N / i, значит, число N / i можно не проверять

Реализация

```
i := 1;
while (sqr(i) <= N) do begin
    if (N mod i = 0) then begin
        writeln(i);
        writeln(N div i);
    end;
    inc(i);
end;
```

Реализация

```
i = 1
while i ** 2 <= N:
    if N % i == 0:
        print(i)
        print(N // i)
    i += 1
```

Реализация

```
for(int i = 1; i * i <= N; ++i){
    if (N % i == 0){
        cout << i << endl;
        cout << N / i << endl;
    }
}
```

Проблема

Проблема

Что будет, если N = 9?

Проблема

Что будет, если N = 9?

i = 1: выведем 1 и 9

Проблема

Что будет, если N = 9?

i = 1: выведем 1 и 9

i = 2: ничего не выводим

Проблема

Что будет, если N = 9?

i = 1: выведем 1 и 9

i = 2: ничего не выводим

i = 3: выведем 3 и 3

Как исправить?

Как исправить?

Дополнительной проверкой:

Как исправить?

Дополнительной проверкой:

Если N делится на i и квадрат i равен N, то выведем только i

Реализация

```
i := 1;
while (sqr(i) <= N) do begin
    if (N mod i = 0) then begin
        writeln(i);
        if (sqr(i) <> N) then
            writeln(N div i);
    end;
    inc(i);
end;
```

Реализация

```
i = 1
while i ** 2 <= N:
    if N % i == 0:
        print(i)
        if i ** 2 != N:
            print(N // i)
    i += 1
```

Реализация

```
for(int i = 1; i * i <= N; ++i){
    if (N % i == 0){
        cout << i << endl;
        if (i * i != N){
            cout << N / i << endl;
        }
    }
}
```

Разложение на простые делители

Пример разложения на простые делители:

$504 = 2^3 * 3^2 * 7$

Разложение на простые делители

Воспользуемся алгоритмом из 6-7 класса

504 | 2

Разложение на простые делители

Воспользуемся алгоритмом из 6-7 класса

504 | 2

252 | 2

Разложение на простые делители

Воспользуемся алгоритмом из 6-7 класса

504 | 2

252 | 2

126 | 2

Разложение на простые делители

Воспользуемся алгоритмом из 6-7 класса

504 | 2

252 | 2

126 | 2

63 | 3

Разложение на простые делители

Воспользуемся алгоритмом из 6-7 класса

504 | 2

252 | 2

126 | 2

63 | 3

21 | 3

Разложение на простые делители

Воспользуемся алгоритмом из 6-7 класса

504 | 2

252 | 2

126 | 2

63 | 3

21 | 3

7 | 7

Пример 2

98 | 2

Пример 2

98 | 2

49 | 7

Пример 2

98 | 2

49 | 7

7 | 7

Пример 2

98 | 2

49 | 7

7 | 7

1

Алгоритм

В отдельную переменную T положим N.

Алгоритм

В отдельную переменную T положим N.

Пробежимся от 2 до корня из T.

Алгоритм

В отдельную переменную T положим N.

Пробежимся от 2 до корня из T.

Если T делится на i, будем на него делить,

пока можем.

Проверим, больше ли T единицы

Алгоритм

В отдельную переменную T положим N.

Пробежимся от 2 до корня из T.

Если T делится на i, будем на него делить, пока можем.

Проверим, больше ли T единицы, если больше, то выведем T.

Реализация

```
T := N;
i := 2;
while (sqr(i) <= T) do begin
    if (T mod i = 0) then begin
        while (T mod i = 0) do begin
            write(i, ' ');
            T := T div i;
        end;
    end;
    inc(i);
end;
if (T > 1) then
    write(T);
```

Реализация

```
T = N
i = 2
while i ** 2 <= T:
    if T % i == 0:
        while T % i == 0:
            print(i, end=' ')
            T = T // i
    i += 1
if T > 1:
    print(T)
```

Реализация

```
T = N;
for(int i = 2; i * i <= T; ++i){
    if (T % i == 0){
        while (T % i == 0){
            cout << i << " ";
            T /= i;
        }
    }
}
if (T > 1){
    cout << T;
}
```

Решето Эратосфена

Решето Эратосфена

Дано число N. Найти все простые числа от 1 до N.

Решето Эратосфена

Дано число N. Найти все простые числа от 1 до N.

Алгоритм

1. Выписать подряд все целые числа от двух до n (2, 3, 4, …, n).
2. Пусть переменная p изначально равна двум — первому простому числу.
3. Зачеркнуть в списке числа от 2p до n, считая шагами по p (это будут числа, кратные p: 2p, 3p, 4p, …).
4. Найти первое незачёркнутое число в списке, большее чем p, и присвоить значению переменной p это число.
5. Повторять шаги 3 и 4, пока возможно.

Пример

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100
101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120

Prime numbers

Реализация

```
var prime: array [0..1000000] of boolean;
    i, j, n: integer;
begin
    read(n);
    for i := 0 to n do
        prime[i] := True;
    prime[0] := False;
    prime[1] := False;
    for i := 2 to n do begin
        if (prime[i]) then begin
            j := 2 * i;
            while (j <= n) do begin
                prime[j] := false;
                inc(j, i);
            end
        end
    end;
    for i := 2 to n do begin
        if (prime[i]) then
            write(i, ' ');
    end;
end.
```

Реализация

```
n = int(input())
prime = [True] * (n + 1)
prime[0] = prime[1] = False
for i in range(2, n + 1):
    if prime[i]:
        j = 2 * i
        while j <= n:
            prime[j] = False
            j += i
for i in range(2, n + 1):
    if prime[i]:
        print(i, end=' ')
```

Реализация

```
int n;
cin >> n;
vector <int> prime(n + 1, 1);
prime[0] = prime[1] = 0;
for(int i = 2; i <= n; ++i){
    if (prime[i]){
        for(int j = 2 * i; j <= n; j += i){
            prime[j] = 0;
        }
    }
}
for(int i = 2; i <= n; ++i){
    if (prime[i]){
        cout << i << " ";
    }
}
```

Пример

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100
101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120

Prime numbers

Оптимизация

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

2: 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, …

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

2: 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, …

3: 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, …

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

2: 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, …

3: 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, …

5: 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, …

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

2: 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, …

3: 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, …

5: 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, …

7: 7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, …

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

2: 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, …

3: 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, …

5: 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, …

7: 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, …

Оптимизация

Выпишем для всех простых чисел те, которые на них делятся

2: 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, …
3: 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, …
5: 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, …
7: 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, …

Бежим не с 2p, а с p^2
(В C++ и Pascal следует использовать типы long long и Int64 соответственно)